CHAPTER 4
IMPLEMENTATION

Implementation translates theoretical architectural models, domain abstractions, and formal software requirement specifications into an integrated, production-grade software system. The HireHub codebase demonstrates high-standard full-stack software engineering practices, incorporating an ultra-responsive React 19 Single Page Application, an asynchronous Node.js and Express RESTful API gateway, a cloud-hosted MongoDB persistent document cluster, and a resilient dual-engine artificial intelligence matching pipeline. This chapter delivers an exhaustive technical discourse detailing the operational implementation of each subsystem, from client route rendering to database normalization and automated algorithm execution.

4.1 Architectural Overview and Technology Stack Implementation

The implementation of HireHub: AI-Powered Local Employment Marketplace operationalizes a decoupled, multi-tier client-server architectural pattern that enforces strict separation of concerns among the presentation layer, the API gateway, the persistent database, and an intelligent analytical processing engine. The presentation tier is constructed as a high-performance Single Page Application (SPA) utilizing React 19, built with Vite for rapid module bundling, tree-shaking, and minification, and styled using Tailwind CSS for a fully responsive, utility-first UI design system. The backend tier operates on the Node.js asynchronous runtime and the Express.js framework, establishing an event-driven RESTful API service. Persistent data storage is managed by MongoDB (provisioned on a MongoDB Atlas cloud replica set and locally configurable), accessed and manipulated through the Mongoose Object Data Modeling (ODM) library. To provide intelligent recruitment capability while guaranteeing platform reliability, the system integrates a dual-engine artificial intelligence subsystem combining Google Gemini 1.5 Flash (via the official @google/genai API SDK) with an in-process, deterministic fallback engine (aiFallbackEngine.js).

4.2 Client-Side Presentation Tier Implementation

The user interface of HireHub is engineered as a component-driven Single Page Application structured around reusable, encapsulated UI components and central state contexts. Navigation across views is orchestrated via React Router v6, utilizing higher-order route wrappers (ProtectedRoute) that evaluate the client's authentication token and user role before granting access to protected views. Global session state is maintained using React's Context API (AuthContext), which synchronizes the decoded JSON Web Token (JWT) payload, user role ('jobseeker', 'employer', or 'admin'), and profile status across the component tree. Communication with the backend gateway is managed through an asynchronous Axios HTTP client pre-configured with interceptors that automatically attach the cryptographic bearer token (Authorization: Bearer <token>) to outbound requests and capture unauthorized response codes (HTTP 401/403) to prompt clean re-authentication. The frontend segregates user workflows into three dedicated portals: (1) the Job Seeker Portal, featuring an interactive profile builder, file-drag resume uploader, faceted search interface with municipal radius filters, and an explainable AI fit modal; (2) the Employer Portal, offering structured multi-step vacancy creation forms (with Indian Rupee salary bounds and skill taggers), a Kanban-style Applicant Tracking System (ATS) board, automated candidate ranking matrices, and interview booking modals; and (3) the System Administration Console, presenting real-time system telemetrics, recruiter verification workflows, and content moderation tools.

4.3 Server-Side REST API and Security Architecture

The backend application server is initialized via server.js and configured within app.js through a modular pipeline of middleware and route handlers. Security is enforced at the network and application boundaries using standard cryptographic protocols. User passwords submitted during registration are salted and hashed using bcrypt with an algorithmic cost factor of 10 salt rounds, ensuring that plaintext credentials are never persisted or exposed. Authentication is entirely stateless: upon successful credential verification, the authentication controller (authController.js) generates an RFC 7519-compliant JSON Web Token signed with a 256-bit server secret and configured with a 24-hour expiration lifetime. Incoming requests to restricted endpoints are intercepted by two sequential middleware guards: verifyToken, which decrypts and validates the cryptographic token signature from the HTTP request headers, and verifyRole, which inspects the decoded role claim to prevent horizontal and vertical privilege escalation. To harden the API against common web vulnerabilities, the server incorporates express-rate-limit to restrict brute-force credential stuffing attempts (limiting authentication routes to 10 requests per 15-minute window) and express-mongo-sanitize to strip reserved MongoDB operator characters ('$', '.') from request bodies, mitigating NoSQL query injection attacks.

4.4 Document Ingestion and Automated Resume Parsing Pipeline

The resume ingestion subsystem bridges unstructured candidate documents with structured database schemas. File uploads are managed using Multer multipart middleware, which enforces a strict 5 megabyte payload ceiling and whitelists accepted MIME types (application/pdf and application/vnd.openxmlformats-officedocument.wordprocessingml.document), rejecting unapproved binary or executable formats immediately at the HTTP boundary. Upon upload, the file buffer is passed to the resumeParser.js utility, where pdf-parse extracts raw, unformatted textual streams from the document. The extracted text undergoes a sequence of natural language cleansing transformations, including punctuation normalization, lowercasing, whitespace stripping, and regex-based tokenization. An automated entity extractor then parses standard resume sections (Education, Technical Competencies, Professional Experience, and Contact Data) by matching against normalized lexical dictionaries and domain-specific taxonomies. The extracted competencies are converted into normalized string tokens and saved directly into the candidate's EmployeeProfile and Resume documents in MongoDB, allowing candidates to review, edit, and augment their parsed credentials before dispatching applications.

4.5 Dual-Engine AI Compatibility Matching Implementation

The matching engine represents the core algorithmic innovation of HireHub, operating as a fault-tolerant, dual-engine cognitive pipeline designed to score candidate-vacancy compatibility objectively. When an application is submitted, the matching orchestrator compiles a structured payload containing the candidate's parsed skills, professional summary, and experience years alongside the target job's mandatory requirements, description, and municipal location. In normal operation, the system invokes the Primary Cloud Engine, leveraging Google Gemini 1.5 Flash through the official Google Generative AI SDK (@google/genai). A carefully calibrated, few-shot prompt directs the model to perform deep semantic reasoning, comparing conceptual qualifications, identifying transferable skills, and returning a strict JSON object containing a composite score (0.0 to 100.0) accompanied by qualitative textual breakdowns of candidate Strengths and identified Skill Gaps. To overcome the vulnerabilities of external cloud APIs -- specifically rate-limit throttling (HTTP 429), latency spikes, and network disconnects -- the orchestrator implements an in-process circuit breaker that switches seamlessly to the Autonomous Local Fallback Engine (aiFallbackEngine.js). The local fallback engine executes entirely in-memory without external network dependencies, calculating: (1) contextual semantic relevance using a TF-IDF vector space model evaluated via cosine similarity; (2) core technical skill overlap using the Jaccard similarity coefficient augmented by fuzzy substring token matching; (3) experience seniority alignment; and (4) municipal district proximity.

Composite AI Match Formula:

$$S_{composite} = 0.40 \cdot S_{skills} + 0.25 \cdot S_{experience} + 0.25 \cdot S_{semantic} + 0.10 \cdot S_{location}$$

(Equation 4.1: Weighted Multi-Factor Compatibility Score)

Executing in less than 15 ms, this deterministic fallback pipeline guarantees 100% platform availability and ensures that candidate evaluations never halt during third-party cloud service disruptions.

4.6 Applicant Tracking System (ATS) and Interview Coordination Workflows

Recruitment operations are driven by a stateful ATS pipeline controller (applicationController.js) that manages candidate progression through defined lifecycle stages: 'pending', 'reviewed', 'shortlisted', 'interview_scheduled', 'rejected', and 'hired'. When candidates apply, the application record is persisted atomically, foreign keys are linked, and the vacancy's applicantsCount is incremented. On the employer portal, all incoming applications for a specific job listing are retrieved and dynamically arranged by an automated candidate ranking matrix that sorts applicants in strict descending order of composite AI match scores. Recruiter interfaces render visual compatibility badges, highlighted competency matches, and direct access to view or download full resumes. When a recruiter shortlists a candidate, the platform enables direct interview coordination through interviewController.js. The scheduling modal captures the interview date, time slot, duration, meeting format (virtual or in-person), and video conference URL (Google Meet, Microsoft Teams, or Zoom). Persisting the interview document automatically advances the application state to 'interview_scheduled' and triggers in-app notification events visible on the job seeker's personal dashboard.

4.7 Database Implementation and Schema Normalization

Persistent data storage is implemented using MongoDB, leveraging Mongoose schemas to enforce relational constraints, strict type-casting, and default values across document collections. Although MongoDB is fundamentally a NoSQL document database, HireHub enforces rigorous normalization principles (from 1NF through BCNF and 4NF) to prevent redundant data duplication and update anomalies. Core entities are partitioned into discrete collections: users (credentials, role assignments, account status), employee_profiles (work experience, education records, verified skills array), employer_profiles (company details, industry, business verification badges), jobs (titles, responsibilities, bounded compensation in INR, skill tags, status), resumes (raw extracted text, document paths, metadata), applications (applicant state tracking, candidate/job foreign keys), ai_match_scores (overall score, dimensional sub-scores, strengths, gaps), and interviews (calendar coordinates, virtual meeting URLs, evaluation notes). Relational integrity is maintained through Mongoose ObjectId references populated via .populate() queries. To ensure high-speed querying on faceted searches, compound text indices are established across 'title', 'description', 'skillsRequired', and 'location' fields, enabling sub-50 ms search query resolution across thousands of local job listings.

Academic Project Metadata:
Project: HireHub — AI-Powered Local Employment Marketplace
Course: 23AD553 - Mini Project - III : Software Engineering for AI Applications
Institution: Mepco Schlenk Engineering College (Autonomous), Sivakasi
Department: Department of Artificial Intelligence and Data Science
Project Team: Biswajith P N (9517202409015), Vibin Aravind R (95172024090122), Mohammed Ameen A (9517202409067)
Submission Period: Fifth Semester, July 2026 – October 2026